HERDR APP · CODEX CLI · LINUX

분할 터미널에서 기획하고, 문서로 이어 개발합니다

Herdr 한 탭을 여러 칸으로 나누어 Codex CLI를 사용합니다.
각 단계에서 **사용할 칸 → 같은/새 세션 → 저장할 문서 → 보낼 요청 → 확인**을 따라가세요.

처음부터 따라 하는 순서

00 준비 → P1~P5 → 01 개발

기획은 같은 세션, 개발은 문서로 새 세션.
개발 01~07 → 10 기록.
08 GitHub·09 배포는 필요한 경우에만.
화면을 나누는 것과 AI 대화를 나누는 것은 다릅니다.

선택 경로: 할 일 앱 예제 · 앱: 개인 할 일 앱 · 폴더: /home/gon/dev/apps/gon-todo-practice

00에서 문서 폴더부터 준비합니다. 예제의 /home/gon은 자신의 홈 경로로 바꾸세요. '내 기획 문서'에서는 확정한 기능·기술·완료 기준을 사용합니다.

전체 실행 그래프 · 번호를 클릭하세요

00 먼저 → P1~P5 → 아래 01부터 개발 · 번호를 클릭하면 상세 안내

먼저, 무엇을 만들지 함께 알아봅니다

- **P1** 문제와 답변 저장 — 기획 A · 같은 세션
- **P2** 조사 질문·분담 — 기획 A · 같은 세션
- **P3** 출처·근거표 — 기획 A · 같은 세션
- **P4** 선택·검증·보류 — 기획 A · 같은 세션
- **P5** 최종 문서·승인 — 기획 A · 같은 세션

P2 → P1 문제 좁히기 · P3 → P2 조사 범위 조정 · P4 → P3 추가 근거 / P1 재정의
직접 개발 → P5 최종 문서 → 01 새 개발 세션 · 기존 도구 사용/보류 → 기록 후 종료

00에서 분할 터미널·문서 폴더를 먼저 준비합니다. 승인된 기획이 이미 있으면 P1~P5를 다시 할 필요는 없습니다.

A 준비하기

- **00** 분할 터미널 준비 — Herdr · 기획 탭 · 처음에는 쉘
- ↓
- **01** 개발 새 세션 — Herdr · 새 개발 탭 · A
- ↓
- **02** 지침·스킬 확인 — Herdr · 개발 A의 Codex

B 만들기

- **03** 문서 인계 확인 — Herdr · 개발 A의 Codex
- ↓
- **04** 문서·환경 준비 — Herdr · 개발 A + 문서 확인 C
- ↓
- **05** 기능 구현 요청 — Herdr · 개발 A의 Codex

C 확인하기

- **06** 실제 결과 직접 사용 — Herdr · B 서버 / C 검사 + 외부 브라우저
- ↓
- **07** 오류 수정·검토 — Herdr · 개발 A + B/C

실패 ↺ 05 → 06 → 07
로컬 완료 → 10 · 원격 보관 → 08

D 보관·마무리

- **08** GitHub 보관 · 선택 — Herdr · 개발 A + C / GitHub 브라우저
- ↓
- **09** 배포 · 선택 — Herdr · 개발 A + C / 운영 URL 브라우저
- ↓
- **10** 저장·종료·재개 — Herdr · 개발 A + C

08 보관 후 → 10
배포 선택 시에만 09 → 10

헷갈리기 쉬운 세 가지부터 구분하세요

분할 터미널은 화면의 칸, 세션은 Codex와 나눈 대화, 문서는 디스크에 저장한 파일입니다. 칸을 나눴다고 대화가 공유되거나 파일이 복제되지는 않습니다.

① 한 탭을 나누는 방법 · tmux의 pane과 같은 뜻

Herdr의 **space**는 프로젝트 묶음, **tab**은 기획·개발 같은 작업 화면입니다. 탭 안을 나눈 각 터미널이 **pane(분할 터미널)**입니다. Files·Changes는 보조 **패널**이며 아래 A·B·C와 역할이 다릅니다. Herdr 안에서 tmux를 따로 실행할 필요는 없습니다.

설명용 배치도 · 실제 앱 스크린샷 아님 · 한 탭 안의 세 분할 터미널

A · 주 기획 Codex

00에서 새 세션 시작
P1 → P2 → P3 → P4 → P5

같은 대화를 이어갑니다.
통합 문서는 A만 작성합니다.

`docs/research/01~04`
`docs/PROJECT-BRIEF.md`

B · 독립 조사 Codex (선택)

P2에서 범위를 정한 뒤 새 세션.
한 질문과 결과 파일 하나만 담당.

`notes/tool-comparison.md`

C · 파일 확인용 쉘

Codex를 켜지 않습니다.
pwd · ls · less로 실제 파일 확인.

개발 탭 · 설명용 배치도 · 기획 탭을 남기고 새 개발 탭을 만듭니다

A · 개발 Codex

새 세션에서 최종 문서 읽기
인계 확인 → 구현 → 수정

첫 출시 범위는 같은 대화.
완료 뒤 독립된 새 범위는 새 세션.

B · 실행 서버용 쉘

프로젝트가 알려준 서버 명령 실행.
로그를 계속 봅니다. 중지는 이 칸의 Ctrl+C.

C · 검사·문서용 쉘

테스트와 파일 확인.
기준 파일을 바꾸는 작업은 A와 겹치지 않게.

1. F2로 탭 이름을 정합니다. `Ctrl+Shift+O`로 오른쪽을 나눕니다.
2. 새 오른쪽 칸을 클릭한 뒤 `Ctrl+Shift+E`로 아래를 나누면 A / B·C 배치가 됩니다.
3. 분할 경계를 드래그해 크기를 조절합니다. 긴 문서를 읽을 칸을 선택하고 `Ctrl+Shift+Z`로 확대·복귀합니다.
4. 문자 붙여넣기는 이 설치본 기준 `Ctrl+Shift+V`입니다. 다른 칸을 클릭할 때마다 입력 위치와 폴더를 확인합니다.

입력 종류	어디에 넣나요?	예시
쉘 명령	Codex가 실행되지 않은 터미널 칸	`pwd`, `codex resume`
자연어 요청	Codex 대화 입력칸	“P1 문서를 읽고 조사 질문을 정리해줘.”
Codex 명령	Codex 대화 입력칸, 한 줄씩	`/status`, `/rename 프로젝트-기획`

> A·B·C는 안내용 위치 이름입니다. Herdr가 자동으로 이 이름을 붙이는 것은 아닙니다. 칸의 상단 프로그램·폴더와 대화 이름으로 구분하세요. 새로 나눈 칸은 쉘이므로 codex를 실행하기 전에는 두 번째 AI가 아닙니다.

키가 다르면: Ctrl+Shift+P의 명령 팔레트나 Ctrl+Shift+? 도움말에서 실제 단축키를 확인합니다. 이 안내는 설치본 Herdr app 0.1.11의 설정을 확인했습니다. 터미널판 Herdr(TUI)의 `Ctrl+B` 접두키 안내와 섞지 않습니다.

② 같은 세션·새 세션·resume·fork를 고르는 기준

하나의 결과를 끝낼 때까지 같은 대화를 기본으로 합니다. 이 안내의 기획 결과는 승인된 개발 시작 문서, 개발 결과는 합의한 첫 출시 범위입니다. 예제의 R01~R05는 같은 세션에서 하나씩 구현·확인합니다. 할 일 앱 완성 뒤 독립된 달력 기능처럼 새 범위를 시작하면 문서로 새 세션을 엽니다. 프로젝트 전체를 무조건 한 대화에 넣거나 단계마다 새 대화를 만들 필요는 없습니다.

지금 상황	선택	그다음 행동
P1→P2, P3 추가 조사, P4 정정	같은 기획 세션	A에서 계속 질문하고 해당 단계 파일을 갱신합니다.
독립된 자료 조사 한 건	B의 새 세션	P1·P2 파일을 읽히고 지정한 notes 파일 하나만 작성합니다.
P5 확정 뒤 구현을 시작	새 개발 세션 권장	최종 문서와 STATE를 읽혀 요약부터 확인합니다. 기능상 강제 조건은 아닙니다.
방금 만든 기능의 오류 수정	같은 개발 세션	재현 순서·기대·실제·오류를 이어 보냅니다.
어제 중단한 같은 작업	기존 세션 재개	쉘에서 `codex resume` → 이름·폴더·시각으로 선택 → 현재 파일 재확인.
첫 범위 완료 후 독립된 새 범위	새 세션	쉘에서 `codex` 또는 기존 Codex에서 `/new`. STATE·개발 기준 문서를 읽힙니다.
같은 배경으로 다른 접근 비교	fork	대화를 복사한 새 ID입니다. 파일은 자동 복제되지 않습니다. 다른 결과 파일이나 worktree를 정합니다.
같은 작업인데 대화가 길어짐	기록 후 compact	중요 결정을 파일에 저장하고 `/compact`. 같은 대화를 요약해 이어갑니다.

fork 실행: 현재 작업을 마치고 A의 Codex에서 `/quit`로 쉘에 나온 뒤, 같은 프로젝트 폴더에서 `codex fork`를 실행합니다. 목록에서 원본 대화를 골라 새 갈래를 시작합니다. 두 대화가 같은 파일을 함께 수정하지 않게 결과 파일 또는 worktree를 먼저 나눕니다.

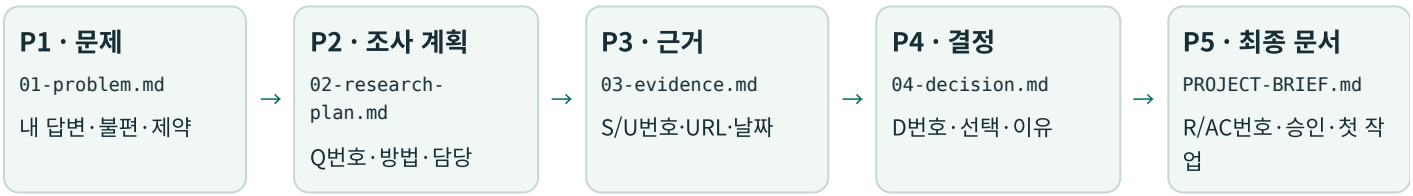
대화 이름과 ID: `/rename 프로젝트-기획`, `/rename 프로젝트-첫기능`으로 구분하고 `/status`에서 상태·식별 정보를 확인합니다. 두 세션이 있는 폴더에서 `--last`를 쓰면 의도와 다른 최근 대화를 고를 수 있으므로 기본은 목록 선택입니다.

무엇을 닫았나요?	무엇이 남나요?	돌아오는 방법
Herdr 앱 창	core가 살아 있으면 터미널·Codex·서버 실행 유지	Herdr를 다시 열고 기존 칸을 선택합니다.
Codex에서 `/quit`	Codex 실행 종료, 저장된 대화와 파일은 별개로 남음	쉘에서 `codex resume`로 선택합니다.
pane 또는 tab 닫기	그 안의 터미널 실행 종료	새 터미널에서 폴더를 확인하고 필요한 프로그램·대화를 재개합니다.
재부팅·core 종료	기존 프로그램 실행은 유지되지 않음. kept pane은 마지막 폴더의 새 쉘로 복원될 수 있음	Codex는 resume, 서버는 실행 명령으로 각각 다시 시작합니다.

> **세션과 파일은 별개입니다.** `/new`로 새 대화를 열어도 폴더의 파일은 그대로입니다. `fork`는 대화의 갈래이며 Git branch/worktree와 다릅니다. 같은 폴더에서 두 Codex가 동시에 같은 파일을 수정하지 않도록 담당을 나누세요. 동일한 Codex 세션을 두 분할 터미널에서 동시에 진행하지 않습니다.

③ P1~P5의 기록이 최종 개발 문서가 되는 과정

대화 전체를 합치는 것이 아니라, 질문 → 근거 → 결정 → 요구사항을 연결합니다. 처음 만든 프로젝트 폴더가 저장 위치입니다. 기존 프로젝트라면 대응하는 기존 문서 이름을 우선합니다.




```
내 프로젝트/
└── docs/
    ├── research/
    │   ├── 01-problem.md
    │   ├── 02-research-plan.md
    │   ├── 03-evidence.md
    │   ├── 04-decision.md
    │   └── notes/tool-comparison.md  ← B가 쓴다면 이 파일만
    ├── PROJECT-BRIEF.md              ← 개발 시작 때 먼저 읽을 문서
    └── STATE.md                      ← 현재 단계·세션 이름·다음 작업
```

확인할 때는 C의 쉘에서 `ls docs/research`로 파일 존재를 보고, `less docs/PROJECT-BRIEF.md`로 내용을 읽습니다. `less`는 읽기 도구이며 `q`로 나옵니다. Files 패널도 파일을 읽어볼 수 있지만 이 설치본에서는 읽기 전용 미리보기입니다. 수정은 A에게 요청하거나 별도 편집기를 사용합니다.

실제 문장은 이렇게 이어집니다

P1·U02/U03: “새로고침 후 남아야 한다. 같은 PC·브라우저에서 쓴다.”

P4·D02: 추가·완료·삭제·새로고침 유지를 포함하고 로그인·동기화는 제외.

P5·R04: 목록과 완료 상태를 같은 브라우저에 저장한다.

P5·AC04: 항목을 추가하고 완료 표시한 뒤 새로고침해도 두 상태가 남는다.

첫 구현 작업: 할 일 추가와 빈 입력 안내부터 구현 → 직접 확인 → 다음 기능.

위 문장은 설명용 예시입니다. 실제 사용자 인터뷰나 완성 앱의 검증 결과가 아닙니다. 실제 프로젝트에서는 내 답변과 확인한 근거로 바꿔야 합니다.

이 예시는 Q03 → U02(새로고침 요구), Q02 → U03(같은 브라우저 제약) → D02 → R04·AC04로 연결됩니다. S01은 미확인이므로 저장 API를 확정하는 근거로 쓰지 않습니다.

최종 문서에는 이 내용이 보여야 합니다

1. **앞쪽 요약:** 무엇을 왜 만드는지, 첫 범위와 제외, 가장 중요한 완료 기준.
2. **세부 요구:** 사용 흐름, R번호별 기능, AC번호별 확인 행동과 기대 결과.
3. **제약과 결정:** 데이터·운영·기술 확정/미정, 근거 링크, 남은 미정과 시작을 막는지 여부.
4. **출발점:** 첫 구현 작업, 승인 상태·실제 날짜·문서 버전, 연결된 STATE.

초안 → 내가 읽고 정정 → 실제 합의 → 승인 기록 순서를 따릅니다. 미정이 있는데 AI가 임의로 “승인 완료”라고 쓰지 않게 합니다. P5의 승인 요청에서 대괄호를 실제 확인 내용으로 바꾸고, 03의 인계 요청을 사용하면 문서로 인계할 수 있습니다.

개발 시작 후 `PRODUCT.md`와 `ARCHITECTURE.md`가 필요하면 04에서 기존 문서와 연결해 만듭니다. PROJECT-BRIEF는 기획 출발점의 근거, PRODUCT는 개발 중 유지할 범위·완료 기준입니다. 범위 변경 때 두 문서에 서로 다른 결론을 방치하지 않고 변경 이유와 현재 기준을 STATE에 연결합니다.

채워진 최종 문서 예시 예시 파일을 읽는 순서 P1~P5 예시 전체 내려받기 빈 최종 문서 양식

단계 00

열 창 · Herdr · 기획 탭 · 처음에는 쉘

Herdr 한 탭을 세 칸으로 나누고 폴더를 준비한다

00 → P1~P5 → 개발 순서입니다. 기획부터 파일을 저장할 폴더를 만듭니다.

> **세션 · 새 기획 세션 시작**
> A에서 처음 한 번 codex를 실행합니다. P1~P5는 이 기획 세션을 계속 씁니다.

> **쉽게 이해하기** pane은 한 탭을 나눈 터미널 칸입니다. Files·Changes 같은 보조 패널과 구분합니다. 한 칸에서 Codex가 실행 중일 때 그 칸에 codex를 또 입력하지 않습니다.

① 이 순서로 누르세요

1. Ubuntu 앱 목록에서 Herdr를 엽니다. 다른 프로젝트 화면이면 Ctrl+Shift+N으로 새 space를 만들고 Shift+F2로 프로젝트 이름을 붙입니다. 로컬 컴퓨터의 space를 사용합니다.
2. F2로 첫 탭을 “기획”이라 이름 붙입니다. Ctrl+Shift+O로 오른쪽을 나누고, 새 오른쪽 칸을 클릭한 뒤 Ctrl+Shift+E로 아래를 한 번 더 나눕니다.
3. 왼쪽 큰 칸을 A(주 기획), 오른쪽 위를 B(별도 조사), 오른쪽 아래를 C(파일 확인용 쉘)라고 생각합니다. A·B·C는 이 안내서의 위치 표시이며 실제 pane 이름은 아닙니다.
4. A를 클릭하고 아래 “A 쉘” 명령을 순서대로 실행합니다. 프로젝트 폴더를 만들고 이동한 뒤 Codex를 시작합니다. 기존 프로젝트는 새 폴더 대신 실제 프로젝트 경로를 사용합니다.
5. B·C는 같은 탭 안의 분할 터미널입니다. 각 쉘에서 아래 B·C 명령으로 프로젝트 폴더에 이동한 뒤 pwd로 확인합니다. 폴더가 자동으로 같아졌다고 가정하지 않습니다.
6. A에서 /rename으로 기획 대화 이름을 붙이고 /status를 확인합니다. 아래 초기 확인 요청을 보낸 뒤 P1로 갑니다. B는 P2에서 필요할 때만 Codex를 시작합니다.

② 이렇게 설정하세요

이름 구분	space=프로젝트 / tab=기획·개발 / pane=분할 터미널
문자 붙여넣기	이 설치본의 Herdr: Ctrl+Shift+V
확대·복귀	분할 칸 클릭 → Ctrl+Shift+Z
여러 AI를 쓸 때	대화는 별개·같은 폴더의 파일은 공유

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/STATE.md`	A 작성	기획 시작 상태
`docs/research/notes/`	A 준비	독립 조사 결과를 둘 폴더

③ 표시된 입력칸에 복사하세요

A 쉘 · 폴더 준비 후 Codex 시작

쉘에 붙여 넣습니다. Codex의 대화 입력칸에는 넣지 않습니다.

```
mkdir -p '/home/gon/dev/apps/gon-todo-practice'
cd '/home/gon/dev/apps/gon-todo-practice'
pwd
codex --search --yolo
```

B·C 쉘 · 각각 프로젝트 폴더 확인

쉘에 붙여 넣습니다. Codex의 대화 입력칸에는 넣지 않습니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
pwd
```

Codex · 대화 이름 붙이기

Codex 입력칸에 이 명령 하나만 보냅니다.

```
/rename 개인 할 일 앱-기획
```

Codex · 현재 상태 확인

Codex 입력칸에 이 명령 하나만 보냅니다.

```
/status
```

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 기획 탭 · 처음에는 쉘

```
이 세션은 개인 할 일 앱의 주 기획 A야. 작업 폴더 /home/gon/dev/apps/gon-todo-practice가 맞는지 실제로 확인해줘.
기존 AGENTS.md·기획 문서·미커밋 변경을 먼저 읽고 보존해.
이번에는 기획 문서만 다룬다. 코드·패키지·배포 설정은 아직 만들지 마.
기존 문서 체계가 있으면 그 위치를 우선하고 아래 이름과의 대응을 알려줘. 없으면 docs/research/notes/와 docs/STATE.md만 준비해.
STATE에는 현재 단계=기획 시작, 최종 문서=아직 없음, 다음 행동=P1 문답이라고 써줘.
기획 통합 문서는 A만 수정하고 별도 조사 B는 지정한 notes 파일만 쓰는 원칙을 기록해.
실제로 확인한 폴더와 만든 파일을 알려준 뒤 내 P1 요청을 기다려.
```

④ 직접 확인한 항목만 체크

- [ ] 기획 탭 안에 A·B·C 분할 터미널이 있다.
- [ ] A의 /status와 C의 pwd가 의도한 프로젝트를 가리킨다.
- [ ] 기존 파일은 보존되고 STATE.md에 현재 기획 상태가 있다.

막히면 이렇게 하세요

명령을 넣었는데 AI가 설명만 하면 Codex 입력칸에 쉘 명령을 넣은 것입니다. C의 쉘에서 실행하거나 Codex에 “실제로 실행해줘”라고 구분해 요청합니다. codex 명령이 없거나 로그인 화면이면 먼저 설치·로그인을 마칩니다.

처음 기획한다면 P1. 이미 승인된 최종 문서가 있으면 01로 이동합니다.

단계 P1

열 창 · Herdr · 기획 탭 · A 분할 터미널의 Codex

질문에 답하고 문제 정의를 파일로 남긴다

AI의 질문·내 답변에서 이번 프로젝트의 문제와 제약을 찾습니다.

> **세션 · 같은 세션 유지**
> 00에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

> **쉽게 이해하기** 채팅은 대화 기록이고, Markdown(.md) 파일은 프로젝트 폴더에 남는 글 문서입니다. AI가 "정리했습니다"라고 말한 것만으로 저장된 것은 아닙니다.

① 이 순서로 누르세요

1. 00에서 프로젝트 폴더와 기획 세션을 먼저 준비합니다. A를 클릭해 아래 요청문을 보냅니다.
2. 질문은 한 번에 1~3개씩 받습니다. 모르면 "미정", 추측이면 "가정"이라고 답합니다. AI가 임의로 내 답을 채우지 않게 합니다.
3. AI가 요약한 대상·불편·원하는 변화·제약을 읽고 잘못 이해한 부분을 바로잡습니다. 아직 기능이나 기술을 급하게 확정하지 않습니다.
4. 대화가 한 덩어리 끝날 때 01-problem.md에 실제 저장하도록 합니다. C의 파일 확인 명령 또는 Files 패널로 파일 내용을 엽니다.
5. 누가 어떤 상황에서 무엇 때문에 불편한지 한 문장으로 설명되면 P2로 갑니다. 답변이 부족하면 이 세션에서 계속 문답합니다.

② 이렇게 설정하세요

입력	한 문장 아이디어 + 실제 겪은 불편
저장	docs/research/01-problem.md
문서 상태	초안 → 사용자 확인
이번에 미뤄도 되는 것	세부 기술·전체 기능·이름

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/research/01-problem.md`	A 작성	질문과 답변·문제·제약·미정

③ 표시된 입력칸에 복사하세요

C · 저장된 문제 정의 읽기

C의 쉘에서 실행합니다. 파일이 없으면 P1 저장부터 확인합니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
sed -n '1,160p' docs/research/01-problem.md
```

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 기획 탭 · A 분할 터미널의 Codex

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고, 연결된 비공개 또는 공개 허용된 저장소가 있으면 협의·근거를 GitHub 이슈/Discussion에 저장해. 없으면 로컬에 보존하고 pending_sync로 남겨. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
개인 할 일 앱 기획을 시작하자. 현재 프로젝트 폴더는 /home/gon/dev/apps/gon-todo-practice야.
내 아이디어: [실제로 하고 싶은 일 한 문장으로 바꾸기]
실제로 불편했던 상황: [내 경험으로 바꾸기]
대상 사용자·현재 방법·불편·원하는 변화·시간/예산/환경 제약을 질문해줘.
한 번에 중요한 질문 1~3개만 하고 내 답을 기다려. 답하지 않은 내용을 지어내지 마.
문답이 한 덩어리 끝날 때마다 docs/research/01-problem.md를 갱신해줘.
질문/내 답변, 확인한 사실, 가정, 미정, 문제 한 문장, 다음 조사 질문을 구분해.
이 단계에서는 코드 구현과 도구 설치를 시작하지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.

파일이 실제로 저장됐는지 확인

A · 같은 기획 세션

지금까지 내가 답하고 정정한 내용으로 docs/research/01-problem.md를 갱신해줘. 저장한 파일을 다시 읽고 절대 경로·핵심 내용·아직 미정인 내용을 알려줘. 다음 단계는 시작하지 마.

AI와 이렇게 주고받으세요 · 설명용 예시

나: 앱을 만들고 싶어.
AI: 누가 어떤 순간에 불편한가요?
나: 내가 연습용 할 일을 자주 잊어. 여러 기기 동기화는 필요 없어.
기록: “개인 학습용”은 결정, “다른 사람도 필요하다”는 아직 가정.

④ 직접 확인한 항목만 체크

- [ ] 01-problem.md를 열어 내 실제 답변과 다른 내용이 없는지 확인했다.
- [ ] 문제 한 문장·대상·제약·미정이 구분되어 있다.

막히면 이렇게 하세요

AI 답변만 보이고 파일이 없으면 아래 “저장 확인 요청”을 보냅니다. 범위가 너무 크면 사용자 한 명·장면 한 개로 좁힙니다.

같은 A 세션에서 P2. 기술을 정하지 못했어도 조사 질문이 명확하면 진행합니다.

단계 P2

열 창 · Herdr · 기획 탭 · A 분할 터미널의 Codex

질문별 조사 방법과 저장 담당을 정한다

"많이 조사" 대신 무엇을 확인하면 결정할 수 있는지 정합니다.

세션 · 같은 세션 유지

00에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

쉽게 이해하기 분할 터미널을 하나 더 만들면 새 쉘이 생깁니다. Codex를 실행해야 두 번째 AI 대화가 시작됩니다. 두 AI는 파일을 공유할 수 있지만 서로의 채팅을 자동으로 읽지는 않습니다.

① 이 순서로 누르세요

1. A에서 P1 파일을 읽게 하고 아래 요청문을 보냅니다. 조사 질문에 Q01, Q02처럼 번호를 붙입니다.
2. 각 질문의 이유, 찾아볼 공식 자료, 직접 확인할 방법, 그만 조사할 기준을 정합니다. 시간 범위도 적습니다.
3. 혼자 순서대로 진행하면 A만 사용합니다. 독립적인 조사 한 건이 있으면 B를 클릭하고 새 Codex를 시작합니다. A 세션을 B에서 resume하지 않습니다.
4. B의 이름을 "프로젝트-도구조사"처럼 구분하고, 읽을 파일과 결과를 쓸 파일 하나를 지정합니다. A와 B가 같은 통합 문서를 동시에 수정하지 않게 합니다.
5. 02-research-plan.md에서 질문·담당·결과 경로를 확인합니다. 질문이 흐리면 P1로 돌아갑니다.
6. 출력 품질이 핵심이면 대표 입력·독립 기대 결과·오류 또는 누락 기준을 정하고 UI 확장 전에 작은 실제 입력 검사를 배치합니다.

② 이렇게 설정하세요

입력	01-problem.md
저장	docs/research/02-research-plan.md
A의 역할	문제·질문·통합 문서 관리
B의 역할(선택)	분리된 질문 조사 → notes/tool-comparison.md

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/research/01-problem.md`	A·B 읽기	조사 배경
`docs/research/02-research-plan.md`	A 작성	질문·방법·담당·완료 기준
`docs/research/notes/tool-comparison.md`	B만 작성(선택)	독립 조사 결과

③ 표시된 입력칸에 복사하세요

B · 독립 조사용 새 Codex 시작

쉘에 붙여 넣습니다. Codex의 대화 입력칸에는 넣지 않습니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
codex --search --yolo
```

B · 조사 세션 이름 붙이기

Codex가 실행된 입력칸에 한 줄씩 입력합니다.

```
/rename 개인 할 일 앱-도구조사
```

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 기획 탭 · A 분할 터미널의 Codex

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고, 연결된 비공개 또는 공개 허용된 저장소가 있으면 협의·근거를 GitHub 이슈/Discussion에 저장해. 없으면 로컬에 보존하고 pending_sync로 남겨. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
docs/research/01-problem.md를 읽고 조사 계획을 작성해줘.
아직 결정할 수 없는 질문마다 Q01부터 번호를 붙여줘.
표 열은 질문 / 이 답이 필요한 결정 / 찾아볼 1차 출처 / 직접 확인할 방법 / 조사 종료 기준 / 담당 / 결과 파일이야.
인터넷 자료로 답할 수 있는 것과 실제 사용자·환경 확인이 필요한 것을 구분해.
우선순위 높은 질문부터 최소 조사 범위와 시간 한도를 제안해.
A는 통합 문서를 담당하고, B에게 줄 독립 질문이 있다면 notes/tool-comparison.md 하나만 쓰도록 분리해.
내가 범위를 확인하면 docs/research/02-research-plan.md에 저장해. 아직 검색 결과를 만들어 넣지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

B에 보낼 독립 조사 요청

B · 새 조사 세션

```
이 세션은 독립 조사 담당 B야. 다른 세션의 대화를 알고 있다고 가정하지 마.
docs/research/01-problem.md와 02-research-plan.md를 실제로 읽어줘.
담당 질문은 [P2에서 정한 Q번호·질문으로 교체]야. 현재 웹 검색으로 원문을 확인해.
결과는 docs/research/notes/tool-comparison.md에만 작성해. 통합 문서와 코드 파일은 수정하지 마.
각 항목에 제목·URL·게시/수정일·확인일·직접 확인한 내용·한계·관련 Q번호를 넣어.
접근하지 못한 자료는 미확인으로 표시하고, 완료 후 저장 경로와 남은 미확인을 알려줘.
```

AI와 이렇게 주고받으세요 · 설명용 예시

Q01: 기존 도구로 해결할 수 있나? → 공식 기능 문서 확인.
Q02: 실제로 자주 불편한가? → 내가 사용한 기록 또는 사용자 확인.
Q03: 브라우저 저장이면 충분한가? → 제약과 데이터 유실 조건 확인.

④ 직접 확인한 항목만 체크

- [ ] 모든 중요한 질문에 답을 찾는 방법과 종료 기준이 있다.
- [ ] 병렬 조사 시 A·B의 작성 파일이 겹치지 않는다.

막히면 이렇게 하세요

B가 A의 대화를 아는 것처럼 답하면 P1·P2 파일을 실제로 읽게 합니다. 같은 파일을 수정했다면 작업을 잠시 멈추고 현재 내용을 확인한 뒤 A가 통합합니다.

조사 범위를 확인한 뒤 P3. B를 사용하지 않으면 A가 같은 질문을 순서대로 조사합니다.

단계 P3

열 창 · Herdr · 기획 탭 · A 분할 터미널의 Codex

조사 결과를 출처가 있는 근거표로 합친다

인터넷에서 찾은 말과 실제로 확인한 사실을 구분합니다.

> **세션 · 같은 세션 유지**
> 00에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

> **쉽게 이해하기** 근거표는 링크 모음이 아닙니다. "이 URL이 어느 질문의 어떤 결론을 뒷받침하는가"가 보여야 합니다. 다른 AI의 답변 자체를 1차 출처로 삼지 않습니다.

① 이 순서로 누르세요

1. A 또는 B가 P2의 질문을 조사합니다. 검색 결과 제목만 믿지 말고 연결된 공식 원문을 확인하도록 합니다.
2. B를 썼다면 저장 완료 응답을 기다립니다. Herdr의 Done은 새 응답이 왔다는 신호이며 근거 검증 완료 판정은 아닙니다.
3. A에게 B 결과 파일과 직접 수집한 자료를 다시 읽게 합니다. 관련 없는 자료를 줄이고 S01, S02처럼 출처 번호를 붙입니다.
4. C 또는 Files에서 03-evidence.md를 엽니다. 중요한 URL을 외부 브라우저로 열어 날짜·문맥·실제 내용을 확인합니다.
5. 접근 제한·출처 충돌·자료 부족은 그대로 남깁니다. 조사 범위가 잘못됐으면 같은 A 세션에서 P2를 수정합니다.

② 이렇게 설정하세요

입력	P2 조사 계획 + notes/* 조사 결과
저장	docs/research/03-evidence.md
출처 번호	S01·S02… / 내 관찰은 U01…
반드시 구분	원문 사실 / 해석 / 미확인

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/research/notes/*.md`	B 작성·A 읽기	아직 통합 전 조사 메모
`docs/research/03-evidence.md`	A 작성	확인한 출처·관찰·추론·빈틈

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 기획 탭 · A 분할 터미널의 Codex

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고, 연결된 비공개 또는 공개 허용된 저장소가 있으면 협의·근거를 GitHub 이슈/Discussion에 저장해. 없으면 로컬에 보존하고 pending_sync로 남겨. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
docs/research/02-research-plan.md와 완료된 docs/research/notes/ 파일을 읽어줘.
높은 우선순위 질문부터 현재 웹의 1차 출처를 직접 열어 확인하고 docs/research/03-evidence.md에 통합해줘.
출처표 열: S번호 / 제목·기관 / URL / 게시·수정일 / 확인일 / 원문에서 확인한 사실 / 관련 Q번호 / 한계.
날짜를 알 수 없으면 알 수 없음으로 쓰고, 읽지 못한 원문을 읽었다고 하지 마. 긴 원문 전체를 복사하지 말고 필요한 짧은 근거와 요약을 남겨.
내 실제 관찰은 U번호로 따로 기록하고, AI 추론·서로 충돌하는 근거·미확인 질문을 별도 구분해.
B의 결과도 검토 없이 확정 근거로 올리지 마. 주요 결론에 필요한 원문 확인 상태를 표시해.
내가 직접 열어 확인할 핵심 링크와 다음 단계에서 판단할 항목을 알려줘.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

AI와 이렇게 주고받으세요 · 설명용 예시

나: 이 자료는 2년 전 가격이네.
AI: 현재 공식 가격표를 다시 확인하고 확인일을 적겠습니다.
기록: 최신 여부가 확인되지 않은 가격은 “미확인”으로 남깁니다.

④ 직접 확인한 항목만 체크

- [ ] 핵심 결론에 연결된 출처·날짜·관련 질문이 있다.
- [ ] 확인 못 한 사실과 출처 충돌을 감추지 않았다.

막히면 이렇게 하세요

검색이 안 되면 /status와 실행 옵션을 확인합니다. 필요하면 A에서 기록을 저장한 뒤 /quit로 Codex를 종료합니다. 같은 A의 쉘에서 codex --yolo resume --search를 실행해 기존 기획 대화를 선택합니다. B·C에서 같은 세션을 동시에 재개하지 않습니다. 검색 가능과 로그인된 유료 자료 접근은 별개입니다.

근거가 충분하면 P4. 부족하면 질문을 좁혀 P2 또는 추가 조사로 돌아갑니다.

단계 P4

열 창 · Herdr · 기획 탭 · A 분할 터미널의 Codex

대안을 비교하고 내 결정을 기록한다

개발할지, 기존 도구를 쓸지, 더 확인할지 판단합니다.

> **세션 · 같은 세션 유지**
> 00에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

> **쉽게 이해하기** "AI 추천"과 "내가 선택"은 다릅니다. AI가 추천했다고 확정으로 쓰지 않고, 내 답변을 받은 뒤 결정 기록을 갱신합니다.

① 이 순서로 누르세요

1. A가 P1~P3를 읽고 기존 도구 사용·작게 직접 개발·보류를 비교하게 합니다. 비교 기준은 내 제약과 목표입니다.
2. 결정을 바꿀 중요한 가정 1~2개를 고릅니다. 직접 해 볼 수 있는 확인 방법과 통과 기준을 정합니다.
3. 사용자는 실제 사용·테스트·대화를 수행하거나 이번에는 못 했다고 알립니다. AI가 인터뷰나 수요 확인을 수행한 것처럼 쓰지 못하게 합니다.
4. 04-decision.md에 D01처럼 결정 번호를 붙이고 선택·이유·참조 S/U번호·제외 범위·미확인·다시 검토할 조건을 저장합니다.
5. 직접 개발을 선택하고 시작을 막는 미정이 해소되면 P5로 갑니다. 근거 부족은 P3, 문제 자체가 달라지면 P1, 기존 도구·보류는 결정 기록 후 마칩니다.

② 이렇게 설정하세요

입력	문제·조사 계획·근거표
저장	docs/research/04-decision.md
진행 선택	직접 개발 / 추가 확인 / 기존 도구 / 보류
검증 기록	누가·언제·무엇을 실제로 했는지

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/research/03-evidence.md`	A 읽기	비교의 근거
`docs/research/04-decision.md`	A 작성	결정·이유·제외·재검토 조건

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 기획 탭 · A 분할 터미널의 Codex

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고, 연결된 비공개 또는 공개 허용된 저장소가 있으면 협의·근거를 GitHub 이슈/Discussion에 저장해. 없으면 로컬에 보존하고 pending_sync로 남겨. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
docs/research/01-problem.md, 02-research-plan.md, 03-evidence.md를 읽고 선택지를 비교해줘.
기존 도구 사용 / 최소 범위 직접 개발 / 보류를 내 목표·시간·비용·운영 부담으로 비교해.
중요한 주장은 S번호 또는 U번호와 연결하고 부족한 근거를 표시해.
결정을 바꿀 가정 1~2개, 직접 검증 방법, 통과 기준을 제안한 뒤 내 실제 확인 결과를 기다려.
직접 확인하지 않은 인터뷰·사용·수요·실험을 지어내지 마.
내 선택을 받으면 docs/research/04-decision.md에 D번호·선택·이유·근거·제외 범위·미확인·재검토 조건을 저장해.
학습 목적의 실습이라면 시장 수요 검증과 구분해. 아직 최종 기획 승인이나 구현 완료로 표시하지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.

AI와 이렇게 주고받으세요 · 설명용 예시

나: 기존 앱이 더 편하지만 React를 배우려는 목적이라 작은 앱을 만들래.
기록 D01: 학습용으로 직접 개발. 판매·사용자 수요는 검증하지 않았음.
제외: 로그인·동기화·상용 운영.

④ 직접 확인한 항목만 체크

- [ ] 04-decision.md의 결정이 내 선택과 일치한다.
- [ ] 실제 검증·가정·미확인과 재검토 조건이 구분되어 있다.

막히면 이렇게 하세요

큰 미정이 남았으면 승인 표시 대신 보류합니다. 개인 학습용 프로젝트와 사용자 수요를 검증한 제품을 구분합니다.

개발 선택 → P5 / 근거 부족 → P3 / 문제 재정의 → P1 / 기존 도구·보류 → 기록 후 종료.

단계 P5

열 창 · Herdr · 기획 탭 · A 분할 터미널의 Codex

자료를 개발 시작 문서 하나로 정리한다

새 개발 세션이 대화 전체를 몰라도 무엇을 만들지 이해하게 합니다.

> **세션 · 같은 세션 유지**
> 00에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

> **쉽게 이해하기** 최종 문서는 "무엇을 왜 만들고, 무엇이면 끝인지"를 정합니다. 세부 코드 설계는 개발 시작 후 구체화할 수 있습니다. 기술 미정 자체는 가능하지만 예산·데이터·필수 기능을 결정할 수 없는 미정은 먼저 풀어야 합니다.

① 이 순서로 누르세요

1. A의 같은 기획 세션에서 P1~P4 파일을 모두 읽고 최종 문서 초안을 만듭니다. 채팅 전문을 이어 붙이는 작업이 아닙니다.
2. docs/PROJECT-BRIEF.md에 목표·사용자·첫 범위·제외·사용 흐름·요구사항·완료 기준·제약·미정·첫 작업을 정리합니다. 주요 결정은 D번호, 근거는 S/U번호로 연결합니다.
3. C에서 최종 문서를 직접 읽습니다. "데이터 저장"처럼 모호한 표현은 "브라우저 새로고침 후 같은 목록이 남음"처럼 확인 가능한 기준으로 고칩니다.
4. 초안을 읽고 수정할 부분을 A에 답합니다. 실제로 합의한 뒤 아래 승인 기록 요청을 보냅니다. 코드로 들어가기 전에 막는 미정이 남으면 보류로 둡니다.
5. 확정한 문서에는 버전·실제 승인일·변경 요약을 기록합니다. docs/STATE.md에는 최종 문서 경로, 기획 세션 이름, 남은 미정과 다음 첫 요청을 적습니다.
6. 이제 기획 탭은 보관하고 개발 탭에서 새 세션을 시작합니다. 새 세션은 최종 문서를 읽어 요약부터 하며, 곧바로 구현하지 않습니다.

② 이렇게 설정하세요

최종 입구	docs/PROJECT-BRIEF.md
근거 보관	docs/research/01~04 문서 + notes/
상태	초안 → 사용자 확인 → 개발 시작 승인 또는 보류
다음 세션	개발 탭의 새 Codex 대화

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/research/01~04 문서`	A 읽기	앞 단계의 사실·근거·결정
`docs/PROJECT-BRIEF.md`	A 작성 → 사용자 확인	개발 시작 때 읽을 최종 입구
`docs/STATE.md`	A 갱신	현재 단계·세션 이름·다음 행동

③ 표시된 입력칸에 복사하세요

C · 최종 문서 열어 읽기

방향키로 읽고 q를 누르면 쉘로 돌아옵니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
less docs/PROJECT-BRIEF.md
```

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 기획 탭 · A 분할 터미널의 Codex

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고, 연결된 비공개 또는 공개 허용된 저장소가 있으면 협의·근거를 GitHub 이슈/Discussion에 저장해. 없으면 로컬에 보존하고 pending_sync로 남겨. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
docs/research/01-problem.md, 02-research-plan.md, 03-evidence.md, 04-decision.md를 실제로 읽어줘.
서로 충돌하는 결정이나 시작을 막는 미정이 있으면 먼저 나에게 확인해.
개발 세션이 처음 읽을 docs/PROJECT-BRIEF.md 초안을 작성해줘.
포함할 내용: 문서 상태·버전·날짜, 한 줄 목표, 사용자와 문제, 첫 출시 포함/제외, 핵심 사용 흐름, 요구사항 R번호, 관찰 가능한 완료 기준 AC번호, 데이터·제약, 기술 확정/미정, 근거와 결정, 남은 미정, 첫 구현 작업.
요구사항은 결정 D번호와 연결하고, 결정은 근거 S/U번호로 연결해. 모르는 내용은 제안 또는 미정으로 남겨.
한 페이지 요약을 맨 앞에 두고 필요한 세부표를 뒤에 붙여. 조사 원문과 notes는 보존하고 상대 링크로 연결해.
초안 상태로 저장한 뒤 내가 읽을 핵심 부분과 승인 전에 답할 질문을 보여줘. 내 확인 없이 승인됨으로 바꾸지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

초안을 읽고 합의한 뒤에만 보내기

A · 같은 기획 세션

```
최종 문서를 읽었고, 아래 내용에 합의했어.
수정/확정 사항: [내가 실제로 확인한 내용으로 교체]
시작을 막는 미정이 남았는지 다시 확인해줘. 없으면 docs/PROJECT-BRIEF.md에 실제 승인일·버전·확정 범위와 변경 요약을 기록해. 남았으면 승인 대신 보류로 기록해.
docs/STATE.md에 최종 문서 경로·기획 세션 이름·남은 미정·개발 세션 첫 요청을 저장해. 세션 ID는 확인할 수 있을 때만 적고 추측하지 마.
기획 원본은 남겨 두고 지금은 코드를 구현하지 마.
```

AI와 이렇게 주고받으세요 · 설명용 예시

초안: “할 일을 저장한다.”
내 정정: “같은 브라우저에서 새로고침하면 목록과 완료 상태가 유지돼야 해. 기기 간 동기화는 제외.”
최종 R04 → AC04: 새로고침 후 목록·완료 상태 일치. 근거는 D02·U02·U03.

④ 직접 확인한 항목만 체크

- [ ] 최종 문서만 읽어도 포함·제외·완료 기준·미정·첫 작업을 알 수 있다.
- [ ] 내가 확인한 내용만 승인으로 기록했고 P1~P4 근거 링크가 열린다.
- [ ] STATE.md에 최종 문서 경로와 다음 개발 세션의 첫 요청이 있다.
- [ ] 외부 환경이 막힌 수용 기준과 그 기준에 실제로 의존하는 작업을 구분하고, 독립적으로 허용된 첫 작업을 정했다.

막히면 이렇게 하세요

자료가 많으면 최종 문서의 앞쪽 요약을 줄이고 상세 근거를 링크합니다. 모순이 발견되면 P4를 먼저 수정한 뒤 최종 문서를 다시 생성합니다.

01에서 개발용 새 세션. 기획 세션을 지우거나 초기화할 필요는 없습니다.

단계 01

열 창 · Herdr · 새 개발 탭 · A

개발 탭에서 새 Codex 세션을 시작한다

P5의 승인된 문서를 읽는 새 대화를 시작합니다.

> **세션 · 새 개발 세션 시작**
>
> 이 안내의 기본은 P5 문서 확정 후 새 개발 대화입니다. 작은 후속 수정은 같은 세션으로 이어가도 되며, 새 세션이 기능상 의무인 것은 아닙니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. P5에서 PROJECT-BRIEF.md가 확정됐는지 확인합니다. 기획 탭은 닫을 필요 없이 남겨 둡니다.
2. 기획 A를 선택한 상태에서 Ctrl+Shift+T로 새 탭을 만들고 F2로 “개발”이라 이름 붙입니다. 새 탭은 새 작업 화면이며 기존 기획 대화가 자동 복사되는 것은 아닙니다.
3. 00과 같이 탭을 세 칸으로 나눕니다. 개발 A는 Codex, B는 실행 서버, C는 검사·문서 확인용 쉘입니다. B·C에서 Codex를 켤 필요는 없습니다.
4. A 쉘에서 실제 경로로 이동하고 아래 명령으로 새 Codex를 시작합니다. /rename으로 “프로젝트-첫기능” 이름을 붙입니다.
5. 아래 요청으로 실제 폴더·기준 문서 위치를 확인합니다. 02의 규칙 확인 뒤 03에서 인계 내용을 대조합니다. 아직 구현하지 않습니다.

② 이렇게 설정하세요

기획 탭	P1~P5 원래 세션 보관
개발 A	새 Codex 세션
개발 B	실행 서버용 쉘
개발 C	검사·문서 확인용 쉘

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/PROJECT-BRIEF.md`	개발 A 읽기	P5에서 확정한 문서
`docs/STATE.md`	개발 A 읽기	현재 상태·다음 작업

③ 표시된 입력칸에 복사하세요

개발 A 쉘 · 새 대화 시작

쉘에 붙여 넣습니다. Codex의 대화 입력칸에는 넣지 않습니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
codex --search --yolo
```

Codex · 대화 이름 붙이기

Codex 입력칸에 이 명령 하나만 보냅니다.

```
/rename 개인 할 일 앱-첫기능
```

Codex · 현재 상태 확인

Codex 입력칸에 이 명령 하나만 보냅니다.

```
/status
```

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 새 개발 탭 · A

```
이 세션은 개인 할 일 앱의 개발 담당이야. 이전 기획 대화를 알고 있다고 가정하지 마.
현재 폴더 /home/gon/dev/apps/gon-todo-practice와 기존 지침·문서·미커밋 변경을 읽기 전용으로 확인해.
docs/PROJECT-BRIEF.md와 docs/STATE.md가 실제로 있는지 알려줘. 기존 프로젝트의 문서 위치가 다르면 대응 경로를 알려줘.
문서가 없거나 읽을 수 없으면 먼저 알려줘. 폴더 이름만 보고 기획을 추측하지 마.
아직 파일을 수정하거나 구현·설치하지 말고 확인 결과와 다음 입력을 기다려.
```

④ 직접 확인한 항목만 체크

- [ ] 개발 A가 새 대화이며 기획 세션은 보존되어 있다.
- [ ] 폴더와 최종 문서 존재를 실제 응답으로 확인했다.

막히면 이렇게 하세요

기획 대화가 나타나면 resume로 열었을 수 있습니다. 상태 문서를 먼저 확인한 뒤 새 codex 실행 또는 /new로 별도 개발 대화를 만듭니다. 같은 세션을 두 칸에서 동시에 진행하지 않습니다.

같은 개발 A에서 02 → 03.

단계 02

열 창 · Herdr · 개발 A의 Codex

CLI의 지침·권한·스킬을 확인한다

앱 설정 화면 대신 현재 CLI 세션에서 적용되는 지침을 확인합니다.

> **세션 · 같은 개발 세션 유지**
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

> **쉽게 이해하기** 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. A에서 /status로 폴더·모델·권한 상태를 확인합니다. Herdr는 터미널을 보여 주며 Codex 모델이나 계정을 자동으로 선택해 주지는 않습니다.
2. 아래 요청으로 실제 읽은 AGENTS.md와 사용 가능한 스킬을 확인합니다. $gon-project-lifecycle, 웹 작업이면 $gon-web-engineering을 사용합니다.
3. 스킬이 없으면 “스킬이 안 보일 때” 입구의 요청으로 로더·경로·비활성 설정부터 점검합니다. 없는 스킬을 사용했다고 하지 않습니다.
4. 명령 실행이나 파일 변경 승인을 받으면 요청한 목적·대상 폴더인지 확인합니다. 인증 정보는 채팅에 붙이지 않고 해당 앱의 로그인 절차에서 입력합니다.

② 이렇게 설정하세요

현재 설정	/status와 실제 읽은 지침
세션 경계	모델·권한은 Codex, 화면 분할은 Herdr
기획·개발 기본	workspace-write / on-request
기존 프로젝트	기존 지침·기술·미커밋 변경 우선

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`AGENTS.md`	읽기	기존 프로젝트 지침

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A의 Codex

```
현재 세션의 프로젝트 지침과 도구를 확인해줘.
실제로 읽은 AGENTS.md 경로, 적용되는 프로젝트 규칙, 사용할 수 있는 스킬을 짧게 알려줘.
$gon-project-lifecycle을 사용할 수 있는지, 웹 작업이면 $gon-web-engineering도 확인해.
스킬을 찾지 못하면 찾았다고 하지 말고 현재 로더 기준으로 이유를 점검해.
기존 파일·수동 변경·기술 선택은 보존하고, 이번 작업에 필요한 규칙만 적용해.
설정 파일·설치본·인증·외부 서비스는 이번 확인만으로 변경하지 마.
```

④ 직접 확인한 항목만 체크

- [ ] 현재 프로젝트 지침과 실제 사용 가능한 스킬을 확인했다.
- [ ] CLI 권한 상태와 현재 폴더를 이해했다.

막히면 이렇게 하세요

Herdr 설정과 Codex 설정을 혼동하지 않습니다. 단축키 변경은 Herdr, 모델·세션 권한·스킬은 Codex에서 확인합니다.

같은 개발 A에서 03 인계 확인.

단계 03

열 창 · Herdr · 개발 A의 Codex

최종 문서를 읽히고 첫 개발 범위를 확인한다

새 세션의 이해가 내가 확정한 문서와 맞는지 대조합니다.

> **세션 · 같은 개발 세션 유지**
>
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. 최종 파일은 docs/PROJECT-BRIEF.md입니다. P1~P4 전체 대화 대신 이 문서 경로를 전달합니다. 필요한 근거만 링크를 따라 읽게 합니다.
2. 아래 요청을 보내 AI가 읽은 목표·포함·제외·완료 기준·미정을 요약하게 합니다. 요약이 틀리면 최종 문서와 비교하며 바로잡습니다.
3. 기술이 미정이면 후보와 운영 부담을 확인하고 선택합니다. React + TypeScript는 학습용 후보입니다. 내 선택을 문서에 기록하기 전에는 확정하지 않으며 이후 단계도 그 실제 결정을 따릅니다.
4. 인계가 맞으면 아래 “인계 확인 뒤 진행” 요청을 보냅니다. 기존 PRODUCT·ARCHITECTURE 문서가 있으면 그 문서를 우선 사용하고 중복 기준을 만들지 않습니다.

② 이렇게 설정하세요

예제의 포함	R01~R05: 추가·완료/해제·삭제·새로고침 유지·빈 입력 안내
예제의 제외	로그인·서버·동기화
최종 문서 역할	기획 당시 목표·근거·승인 범위
개발 기준 문서	기존 문서 우선, 없으면 PRODUCT·ARCHITECTURE

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
docs/PROJECT-BRIEF.md	개발 A 읽기	승인된 기획의 입구
기존 PRODUCT·ARCHITECTURE	개발 A 확인	중복 여부·기술 결정

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A의 Codex

```
$gon-project-lifecycle
/home/gon/dev/apps/gon-todo-practice/docs/PROJECT-BRIEF.md와 docs/STATE.md를 실제로 읽어줘.
이 경로는 할 일 앱 학습 예제야. 문서에 확정된 경우에만 추가·완료/해제·삭제·새로고침 유지, React + TypeScript 프런트 전용 범위를 따른다.
문서가 없거나 미승인·보류 상태면 시작하지 말고 알려줘.
읽은 문서 경로·버전, 목표, 포함/제외, 완료 기준, 미정, 첫 작업을 짧게 요약해줘.
필요한 근거만 docs/research의 연결 문서에서 확인해. 문서에 없는 기능을 추가하지 마.
기술이 미정이면 후보·비용·운영 부담을 제시해 내 선택을 기다려.
이번에는 인계 확인까지만 하고 코드는 아직 구현하지 마.
```

인계 확인 뒤 진행

개발 A · 같은 세션

요약한 인계 내용이 맞아. 방금 합의한 범위와 기술을 기준으로 04의 문서·실행 환경 준비를 진행해줘. 기존 기준 문서를 우선하고, 기준이 여러 파일에서 달라지지 않게 연결해.

④ 직접 확인한 항목만 체크

- [ ] 새 세션의 요약이 최종 문서와 일치한다.
- [ ] 첫 기능과 종료 기준·제외 범위를 확인했다.

막히면 이렇게 하세요

“간단한 앱” 같은 요약만 나오면 R번호·AC번호를 포함해 다시 요약하게 합니다. 최종 문서와 다르면 코드가 아니라 인계부터 고칩니다.

인계 확인 뒤 04. 기획을 바꿨다면 P4·P5부터 결정 기록을 갱신합니다.

단계 04

열 창 · Herdr · 개발 A + 문서 확인 C

문서와 실행 환경을 준비한다

목표를 실제 파일·실행 방법과 연결합니다.

> **세션 · 같은 개발 세션 유지**
>
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

> **쉽게 이해하기** 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. 개발 A에 아래 요청문을 보냅니다. 기존 문서를 읽고 필요한 문서·환경만 준비합니다.
2. docs/PRODUCT.md는 개발 중 유지할 기능·완료 기준, ARCHITECTURE.md는 선택한 구조·기술을 기록합니다. 이미 대응 문서가 있으면 새로 만들지 않습니다.
3. P5의 PROJECT-BRIEF.md는 개발 출발 시점의 승인 근거입니다. 진행 중 범위를 바꾸면 개발 기준 문서·결정 기록·STATE를 함께 갱신하고 이유를 남깁니다.
4. A가 설치·실행 방법을 정리하면 C에서 문서를 엽니다. 기존 패키지 관리자를 바꾸거나 실습 편의 때문에 서버를 추가하지 않는지 확인합니다.

② 이렇게 설정하세요

필요할 때만 작성	docs/PRODUCT.md · docs/ARCHITECTURE.md · docs/STATE.md
기존 프로젝트	기존 문서·패키지 관리자·잠금 파일 유지
기술·저장 방식	03에서 실제 선택한 기술. 미정이면 환경 구성 전에 확인

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/PRODUCT.md 또는 기존 문서`	개발 A 작성	개발 중 유지할 범위·완료 기준
`docs/ARCHITECTURE.md 또는 기존 문서`	개발 A 작성	기술·구조·이유
`docs/STATE.md`	개발 A 갱신	준비 결과·다음 작업

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A + 문서 확인 C

```
$gon-project-lifecycle
$gon-web-engineering
/home/gon/dev/apps/gon-todo-practice에서 승인된 기획과 03에서 합의한 기술을 기준으로 문서·실행 환경을 준비해줘.
기술·플랫폼·저장 방식은 03에서 실제로 합의한 문서만 따라. 미정이면 후보와 최신 근거를 확인하고 내 선택을 기다려. 후보였다는 이유로 확정하지 마.
기존 목표·구조·상태 문서가 있으면 갱신하고, 없을 때만 docs/PRODUCT.md, docs/ARCHITECTURE.md, docs/STATE.md를 만들어줘.
PROJECT-BRIEF의 버전·결정과 연결하고 구현 완료 기준을 PRODUCT에 정리해. 기술 결정은 ARCHITECTURE에 기록해.
기존 패키지 관리자와 미커밋 변경을 보존해.
B 쉘에서 서버를 띄우는 실제 명령·URL과 C에서 검사할 명령을 알려줘. 아직 전체 기능은 구현하지 마.
```

④ 직접 확인한 항목만 체크

- [ ] 기준 문서와 기획 원문이 링크로 연결되어 있다.
- [ ] B 서버 실행·C 검사 명령과 성공 조건을 받았다.

막히면 이렇게 하세요

설치나 기본 검사가 실패하면 오류를 해결한 뒤 05로 갑니다. “현재 오류를 재현하고 원인·수정·재검사 결과를 보여줘”라고 요청합니다.

같은 개발 A에서 05. B·C는 쉘로 둡니다.

단계 05

열 창 · Herdr · 개발 A의 Codex

첫 기능을 구현시킨다

실행되는 최소 앱을 만들고 접속 주소를 받습니다.

> **세션 · 같은 개발 세션 유지**
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. 첫 구현 단위를 하나 정합니다. 예제에서는 할 일 추가부터 시작하고 완료 기준을 붙입니다.
2. A에 아래 요청문을 보냅니다. AI가 작업하는 동안 C에서 변경 파일과 문서를 읽어볼 수 있습니다.
3. A가 같은 파일을 수정하는 동안 다른 Codex에게 그 파일 수정·포맷·브랜치 변경을 맡기지 않습니다.
4. 기능 하나가 끝나면 06에서 직접 사용합니다. 별도 독립 기능의 코드를 정말 동시에 수정해야 할 때만 worktree로 폴더를 분리합니다.

② 이렇게 설정하세요

이번 첫 작업	R01 추가 + R05 빈 입력 안내
나중에 이어갈 것	R02 완료/해제 · R03 삭제 · R04 저장
저장 방식	아직 선택하지 않았다면 R04 구현 전 공식 근거·한계를 확인

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/PRODUCT.md`	읽기	이번 구현의 완료 기준
코드·관련 테스트	개발 A 작성	선택한 첫 기능

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A의 Codex

```
$gon-web-engineering
/home/gon/dev/apps/gon-todo-practice에서 PRODUCT와 ARCHITECTURE의 승인 범위를 읽고 첫 기능부터 구현해줘.
학습 예제의 첫 작업은 할 일 추가·빈 입력 안내야. 문서가 다르면 먼저 확인해.
첫 작업 뒤 06에서 내가 확인한 결과를 받고, 같은 세션에서 R02·R03·R04를 한 단위씩 이어가자. 저장 방식이 미정이면 R04 전에 근거와 내 결정을 확인해.
변경 범위를 작게 유지하고 각 단위의 필요한 검사를 실제 실행해.
내가 B에서 실행하고 06에서 직접 확인할 수 있게 실행 명령·URL·행동·기대 결과를 알려줘.
실행하지 않은 검사는 통과로 쓰지 마.
```

④ 직접 확인한 항목만 체크

- [ ] 첫 기능과 실제 실행한 검사가 구분되어 있다.
- [ ] 다음에 직접 확인할 URL·행동·기대 결과를 받았다.

막히면 이렇게 하세요

서버가 켜지지 않으면 04의 환경 오류를 해결합니다. “구현 완료”라는 말만으로 다음 단계를 통과하지 말고 실제 주소를 받아야 합니다.

06에서 이번 구현분만 확인 → 문제면 07. 첫 범위 안의 다음 R번호는 같은 세션으로 05, 첫 범위 완료 뒤 독립된 새 범위는 10 기록 후 새 세션입니다.

단계 06

열 창 · Herdr · B 서버 / C 검사 + 외부 브라우저

브라우저에서 직접 써본다

설명대로 실제 사용되는지 손으로 확인합니다.

> **세션 · 같은 개발 세션 유지**
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

> **쉽게 이해하기** 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. B의 쉘에서 04가 알려준 실제 서버 명령을 실행합니다. B는 로그를 계속 보여 주므로 Codex를 실행하지 않습니다.
2. 서버가 출력한 실제 URL을 브라우저로 엽니다. 문서에 적힌 포트가 이미 사용 중이면 출력된 주소를 따릅니다.
3. 첫 회차는 R01·R05만 확인합니다: “우유 사기” 추가와 공백 입력 거절. R02·R03·R04는 해당 기능을 구현한 뒤 확인합니다. 첫 범위가 완성되면 추가 → 완료 → 새로고침 유지 → 삭제 → 빈 입력의 전체 흐름을 검사합니다.
4. 서버를 켠 B에 검사 명령을 이어 쓰지 않습니다. C에서 별도 검사 명령을 실행하고 오류를 읽습니다. 테스트가 파일을 쓰면 A의 수정을 멈추거나 범위를 조정합니다.
5. 실패하면 행동·기대·실제·오류를 A에 전달합니다. Herdr의 Done 표시나 테스트 종료 코드만으로 화면 성공을 단정하지 않습니다.

② 이렇게 설정하세요

시험 데이터	학습 예제: 우유 사기
첫 회차	이번에 구현한 R01·R05만 확인
저장 검사	R04 구현 뒤 합의한 저장 범위·한계에 맞춰 확인
체크 표시	미구현은 실패와 구분하고 미구현 상태로 남김

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/PRODUCT.md`의 AC번호	사용자·A 읽기	수동 확인 기준

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · B 서버 / C 검사 + 외부 브라우저

```
직접 확인 결과를 전달할게.
확인한 버전/변경: [가능하면 커밋 또는 변경 내용]
AC번호·행동: [무엇을 눌렀는지]
기대 결과: [원래 되어야 하는 일]
실제 결과: [관찰한 현상]
B 서버 로그 또는 C 검사 오류: [비밀값을 제외한 내용]
이 결과를 바탕으로 원인을 좁혀줘. 아직 확인하지 않은 동작은 미검증으로 남겨줘.
```

④ 직접 확인한 항목만 체크

- [ ] 이번에 구현한 R/AC의 행동과 기대 결과를 실제로 확인했다.
- [ ] 미구현·자동 검사·수동 확인·미검증을 구분했다.

막히면 이렇게 하세요

접속 실패이면 B가 실행 중인지, URL·포트가 맞는지부터 봅니다. 서버 중지는 B를 클릭하고 Ctrl+C입니다. A의 Ctrl+C는 AI 작업에 영향을 주므로 칸을 확인합니다.

오류면 같은 A 세션의 07. 모두 맞아도 범위에 맞는 검토 후 마칩니다.

단계 07

열 창 · Herdr · 개발 A + B/C

실패를 고치고 검토한다

문제가 해결되었고 이번 범위가 끝났다는 근거를 모읍니다.

> **세션 · 같은 개발 세션 유지**
> 같은 기능의 오류 수정은 기존 대화가 문맥을 갖고 있으므로 이어갑니다. 멈췄던 세션이면 resume합니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. 06의 실패를 같은 개발 A에 보냅니다. 증상·재현 순서·기대·실제·로그를 포함합니다.
2. A가 필요한 범위만 수정하면 해당 검사와 같은 화면 동작을 다시 확인합니다. 이 오류를 위해 매번 새 대화를 만들지 않습니다.
3. 프로젝트 지침에서 요구하는 외부 리뷰가 있으면 완료 경계에서 한 번 수행하고 지적을 반영합니다. 별도 같은 모델 대화의 답을 독립 승인이라고 부르지 않습니다.
4. 첫 출시 범위가 끝났으면 로컬 완료는 10, GitHub 보관은 08입니다. 그 범위 안의 다음 R번호가 남았으면 같은 세션으로 05에 돌아갑니다.

② 이렇게 설정하세요

검사 기준	03의 수용 기준 + 06의 실제 사용 결과
리뷰 실패	리뷰 미완료로 기록. 테스트 통과와 구분
다음 선택	로컬 마무리 / GitHub 보관 / 배포

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
관련 코드·검사	개발 A 수정	같은 첫 기능의 회귀 수정

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A + B/C

```
방금 전달한 실패를 같은 개발 범위에서 재현하고 원인을 좁혀 수정해줘.
관련 없는 파일과 수동 변경은 보존해. 필요한 검사와 실제 화면 확인 결과를 구분해 보고해.
현재 지침이 요구하는 완료 경계 리뷰가 있으면 그 기준을 따르고, 실행 모델·성공 여부를 확인해.
실패·미실행을 통과로 표시하지 마. 남은 문제와 이제 끝낼 수 있는 범위를 알려줘.
```

④ 직접 확인한 항목만 체크

- [ ] 필수 기능이 실제로 동작한다.
- [ ] 필수 검사·리뷰의 성공/실패/미실행 상태를 안다.

막히면 이렇게 하세요

리뷰 도구가 실패하면 AI에게 정확한 차단 사유와 남은 작업을 기록하게 합니다. 완료가 필요한 필수 검사가 남아 있으면 통과로 체크하지 않습니다.

첫 범위 안의 다음 R번호 → 같은 세션 05 / 범위 완료 → 10 또는 08 / 독립된 새 범위 → 10 기록 뒤 새 세션.

단계 08

열 창 · Herdr · 개발 A + C / GitHub 브라우저

코드를 GitHub에 보관한다

선택 단계입니다. 이 컴퓨터 밖에도 코드와 변경 이력을 보관합니다.

> **세션 · 같은 개발 세션 유지**
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. 기준 이슈·저장소가 있는지 확인하고 보관할 변경 목적을 정합니다. 기존 미커밋 변경을 한꺼번에 포함하지 않습니다.
2. A에 GitHub 보관을 요청하고 실제로 수행된 커밋·push·PR·CI를 각각 확인합니다.
3. C 또는 GitHub에서 저장된 변경과 CI 결과를 다시 확인합니다. 로컬 commit만 있고 push가 없으면 원격 보관 완료가 아닙니다.
4. 원격 보관만 하면 10으로 갑니다. 실제 서비스 배포가 필요할 때만 09로 갑니다.

② 이렇게 설정하세요

저장소 이름	gon-todo-practice
공개 범위	Private
새 저장소	초기 기본 브랜치가 없으면 초기 커밋·push로 시작 가능
기존 저장소	작업 브랜치·PR·필수 CI·정상 병합

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/STATE.md` 또는 기준 이슈	개발 A 갱신	커밋·PR·CI 링크와 미완료

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A + C / GitHub 브라우저

```
검증한 /home/gon/dev/apps/gon-todo-practice의 이번 작업을 GitHub에 보관해줘.
소유자: [GitHub 소유자]
새 저장소가 필요하면 이름은 gon-todo-practice, 공개 범위는 Private이야.
기존 remote가 있으면 대상부터 확인하고 임의로 교체하지 마.
관련 변경만 검토·커밋하고 비밀값과 환경 파일은 제외해줘.
새 저장소의 초기 커밋은 적절히 push하고, 기존 저장소는 브랜치·PR을 사용해줘.
필수 CI가 있으면 최신 커밋 결과를 확인하고 통과한 뒤 정상 병합까지 진행해줘.
CI가 없으면 없음으로 표시해줘. 저장소·커밋·PR·병합 상태를 각각 알려줘.
실제 서비스 배포는 이번 요청에 포함하지 않아.
```

④ 직접 확인한 항목만 체크

- [ ] 원격 저장소에서 이번 커밋의 파일을 볼 수 있다.
- [ ] PR·CI·병합 상태가 각각 명확하다.

막히면 이렇게 하세요

인증 또는 저장소 권한이 없으면 로컬 커밋과 원격 업로드 상태를 구분해 기록합니다. CI가 빨간색이면 Codex에 실패 링크를 주고 원인 수정과 재검사를 요청합니다.

보관만 완료 → 10 / 실제 서비스 배포 선택 → 09.

단계 09

열 창 · Herdr · 개발 A + C / 운영 URL 브라우저

배포 대상을 정하고 공개한다

선택 단계입니다. 대상이 정해진 뒤 실제 배포를 요청합니다.

> **세션 · 같은 개발 세션 유지**
> 합의한 첫 출시 범위의 구현·확인·수정은 같은 개발 대화에서 이어갑니다. 예제 R01~R05는 이 한 세션 안에서 작은 단위로 진행합니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. 배포할 서비스·대상·계정·비용·데이터 영향·되돌리는 방법을 A에게 확인시킵니다.
2. 준비 결과를 보고 실제 배포 요청을 보냅니다. 배포 권한·인증은 해당 서비스의 안전한 경로에서 처리합니다.
3. 실제 배포가 끝나면 운영 URL에서 필요한 기능과 상태를 확인합니다. 로컬 화면·빌드 성공·HTTP 200을 운영 기능 성공과 구분합니다.
4. 기록 후 서버를 종료할 때는 해당 B 쉘만 선택합니다. 로컬 개발 서버 종료와 운영 서비스 종료는 다릅니다.

② 이렇게 설정하세요

먼저 할 일	배포 준비 및 대상 확인
실제 배포 요청	대상·범위 확정 후 별도 요청문 사용
로컬 데이터	새 배포 주소로 자동 이동하지 않음. 주소별 별도 저장

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/STATE.md` 또는 배포 기록	개발 A 갱신	대상·시간·버전·동작 증거·되돌리기

③ 표시된 입력칸에 복사하세요

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A + C / 운영 URL 브라우저

```
이번 앱의 배포 준비를 해줘. 지금은 실제 운영 환경을 변경하지 마.
기존 배포 설정·CI/CD와 가능한 대상을 확인해줘.
대상이 미정이면 정적 웹앱에 맞는 후보와 필요한 계정·비용을 설명해줘.
대상·예상 URL·설정·배포 revision·검사·되돌리는 방법을 구체화해줘.
이미 알려진 서버 주소라도 현재 접속 대상과 환경을 다시 확인해줘.
검토 가능한 준비 결과와 내가 결정할 항목을 보여줘.
```

④ 직접 확인한 항목만 체크

- [ ] 배포할 실제 환경·서비스·URL을 확정했다.
- [ ] 실제 배포를 요청했고 해당 버전의 배포 결과를 받았다.
- [ ] 운영 URL에서 핵심 사용자 동작을 직접 확인했다.

막히면 이렇게 하세요

대상이 없으면 “배포 준비/대상 미정”으로 끝내고 10에 기록합니다. 운영 인증이 없어 화면을 확인하지 못하면 “배포됨/사용자 흐름 미검증”처럼 나누어 기록합니다.

10에 실제 배포 결과·미검증·다음 작업을 기록합니다.

단계 10

열 창 · Herdr · 개발 A + C

기록을 저장하고 다음 세션을 고른다

창·프로세스·대화·파일을 구분하고 안전하게 다음 작업을 잇습니다.

> **세션 · 종료 전 기록 → 다음 목적에 맞게 선택**
>
> /new와 /quit는 선택지입니다. 두 명령을 연달아 실행할 필요가 없습니다. /compact는 같은 대화를 요약하는 기능이며 파일 저장을 대신하지 않습니다.

쉽게 이해하기 각 단계의 요청문은 표시된 Codex 입력칸에 보냅니다. 명령을 실제 실행한 결과와 설명을 구분해 확인합니다.

① 이 순서로 누르세요

1. A에 아래 요청문을 보내 실제 변경·검사·남은 문제·다음 첫 작업을 STATE에 저장합니다. /status에서 세션 이름·ID를 확인해 기록할 수 있습니다.
2. 같은 문제를 내일 이어가면 대화를 유지하거나 같은 세션을 resume합니다. 독립된 새 기능이면 새 세션에서 STATE와 기준 문서를 읽게 합니다.
3. AI 작업이 끝난 뒤 A의 /quit는 Codex를 종료해 쉘로 돌아갑니다. Herdr 창 닫기는 core가 살아 있는 동안 실행을 유지하지만, pane 또는 tab 닫기는 해당 터미널을 종료합니다.
4. PC 재부팅·core 종료 후에는 이전 프로그램이 자동 재실행된다고 생각하지 않습니다. 복원된 쉘의 폴더를 확인하고 codex --yolo resume에서 올바른 대화를 골라 재개합니다. 서버도 별도로 다시 실행합니다.
5. 다른 접근을 비교할 때만 fork를 사용합니다. 새 대화나 fork는 파일을 자동 복제하지 않습니다. 동시 코드 작업의 폴더 격리는 별도 worktree가 필요합니다.

② 이렇게 설정하세요

같은 기능·오류	기존 세션 계속 또는 resume
독립된 새 기능	STATE·최종/개발 기준 문서로 새 세션
대안 실험	fork + 별도 결과 파일, 코드 병렬이면 worktree
대화가 길 때	파일 기록 → /compact, 단계마다 초기화하지 않기

이 단계에서 읽고 남길 파일

파일	누가	남기는 내용
`docs/STATE.md`	개발 A 갱신	실제 결과·다음 작업·세션 식별 정보

③ 표시된 입력칸에 복사하세요

쉘 · 같은 프로젝트의 저장된 대화 고르기

여러 세션이 있으면 목록에서 이름·폴더·시각을 확인합니다. --last는 기본으로 쓰지 않습니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
codex --yolo resume
```

Codex · 같은 대화를 요약해 이어가기

Codex가 실행된 입력칸에 한 줄씩 입력합니다.

```
/compact
```

Codex · 새 대화 선택

새 대화(/new) 또는 종료(/quit) 중 목적에 맞는 하나만 선택합니다.

```
/new
```

Codex · 종료 선택

새 대화(/new) 또는 종료(/quit) 중 목적에 맞는 하나만 선택합니다.

```
/quit
```

쉘 · 저장된 대화를 갈라 다른 접근 비교

현재 A의 작업을 끝내고 /quit로 쉘에 나온 뒤 실행합니다. 목록에서 원본 대화를 고릅니다. 같은 폴더의 파일은 공유되므로 별도 결과 파일이나 worktree를 사용합니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
codex fork
```

아래 요청문은 Codex 대화 입력칸에 보냅니다. 쉘 명령과 구분하세요.

Codex에 보낼 요청문

Herdr · 개발 A + C

```
이번 작업을 마칠 수 있도록 기록해줘.
현재 실제 파일 변경, 실행한 검사와 결과, 수동 확인과 미검증, 커밋·PR·CI·배포 링크가 있으면 그 증거를 정리해.
기존 상태 문서가 있으면 갱신하고 없으면 docs/STATE.md에 저장해.
PROJECT-BRIEF, PRODUCT, ARCHITECTURE 등 실제 기준 문서 경로를 연결해.
다음 작업 하나와 바로 붙여 넣을 재개 요청을 써줘. 이 작업을 같은 세션으로 이어갈지 새 기능 세션으로 시작할지 이유와 함께 적어줘.
내가 별도로 종료하지 않은 서버·외부 작업은 임의로 종료하지 마.
실제 저장한 파일을 다시 읽고 핵심 내용을 알려줘.
```

④ 직접 확인한 항목만 체크

- [ ] STATE.md를 실제로 열어 완료·미완료·다음 행동을 확인했다.
- [ ] 다음에 사용할 기획/개발 세션 이름과 문서 경로를 안다.

막히면 이렇게 하세요

기록 없이 창을 닫았더라도 파일과 세션은 따로 확인합니다. resume 목록에서 이름·시각·폴더로 고르고, 시작 전에 현재 파일·미커밋 변경을 다시 읽게 합니다.

같은 작업은 resume / 새 목표는 새 세션 + STATE·기준 문서.

상황이 다르면 이 입구로 시작하세요

기존 프로젝트라면 어디부터 시작하나요?

새 폴더나 새 기준 문서를 덮어 만들지 않습니다. Herdr space를 실제 프로젝트 폴더에 맞추고 01에서 읽기 전용 점검부터 시작합니다. 기획이 명확하면 P1~P5는 필요한 부분만 합니다.

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
$gon-project-lifecycle
/home/gon/dev/apps/gon-todo-practice의 기존 프로젝트를 이어가자. 기존 지침·문서·미커밋 변경을 먼저 읽어줘.
이번 목표는 [실제 목표]야. PROJECT-BRIEF/PRODUCT/STATE에 대응하는 기존 문서와 현재 상태를 알려줘.
이번 작업에 필요한 기획·검증만 선택하고, 문서가 이미 있으면 새 이름으로 중복 생성하지 마.
읽은 사실과 미확인을 구분해 첫 작업을 제시해.
```

중단한 작업을 같은 세션으로 재개한다

쉘에서 프로젝트 폴더로 이동한 뒤 codex resume로 목록을 엽니다. 이름·시각·폴더가 맞는 대화를 고릅니다. 복원된 대화가 현재 파일 상태까지 자동 보장하지는 않습니다.

쉘 · 저장된 대화 선택

쉘에 붙여 넣습니다. Codex의 대화 입력칸에는 넣지 않습니다.

```
cd '/home/gon/dev/apps/gon-todo-practice'
codex resume
```

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
이전에 하던 같은 작업을 이어가자. 현재 폴더와 docs/STATE.md, 실제 기준 문서, 미커밋 변경을 먼저 읽어줘.
대화 이후 달라진 파일이 있는지 확인하고 완료·미완료·다음 작업 하나를 요약해.
이전 답변만 믿고 이미 한 작업을 반복하거나 현재 변경을 덮어쓰지 마.
기록과 실제 상태가 다르면 이유를 확인한 뒤 이어가자.
```

다른 곳에서 만든 기획 문서를 가져온다

기존 기획 파일을 프로젝트의 적절한 문서 폴더에 넣거나 접근 가능한 경로를 전달합니다. 출처·결정·완료 기준이 충분하면 P1부터 다시 하지 않고 문서 검토 후 01→02→03으로 갑니다.

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
[기존 기획 문서의 실제 경로]를 읽고 이 프로젝트의 최종 인계 자료로 쓸 수 있는지 확인해줘.
목표·대상·범위·제외·완료 기준·근거·결정·미정을 구분하고 부족한 부분만 질문해.
기존 파일을 덮어쓰지 말고 기준 문서 위치를 정리해.
접근하지 못하면 읽었다고 하지 말고 본문이나 내려받은 파일을 요청해.
```

기존 ai-engineering-os 프로젝트

Herdr의 터미널 배치는 같지만 문서 이름·명령·검증 계약은 그 프로젝트 지침을 따릅니다. 이 안내의 docs/ 파일을 그대로 중복 생성하지 않습니다.

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
$gon-project-lifecycle
/home/gon/dev/apps/gon-todo-practice의 ai-engineering-os 지침·PROJECT.md·기존 상태 문서를 먼저 읽어줘.
이 안내의 문제 정의·근거·결정·최종 인계·개발 상태에 대응하는 기존 문서 위치를 매핑해.
기존 계약을 유지하며 필요한 내용만 보완하고 실제 검증 절차로 이어가자.
```

Codex CLI에서 스킬이 안 보인다

Herdr 패널을 바꿔도 Codex 스킬이 설치되는 것은 아닙니다. 현재 CLI의 로더·설정 홈·공통 개인 스킬 경로를 확인합니다.

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
현재 Codex CLI에서 gon-project-lifecycle / gon-web-engineering 스킬이 보이지 않는 이유를 읽기 전용으로 확인해줘.
CODEX_HOME 적용 여부, 현재 로더, ~/.agents/skills와 유효 설정 홈의 skills, 비활성 설정을 확인해.
민감한 설정값은 출력하지 말고 경로·활성 여부·원인만 알려줘.
설치·설정 변경이 필요하면 기존 변경을 보존할 수 있는 구체적인 방법부터 제시해.
```

실제 배포 요청 — 준비 결과를 확인한 뒤

09에서 확인한 대상·영향·검증·되돌리기 계획을 읽고 실제 배포를 원할 때 보냅니다.

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
앞에서 확인한 배포 대상과 범위로 실제 배포를 진행해줘.
승인된 대상·변경·계정만 사용하고 기존 CI/배포 절차를 따라줘.
배포된 버전, 실행 결과, 운영 URL의 실제 동작·상태, 미검증, 되돌리는 방법을 기록해.
준비 완료나 빌드 성공을 실제 배포 완료로 표시하지 마.
```

기존 도구 사용·보류로 마친다

A의 같은 기획 세션에서 결정 기록을 남깁니다. 개발하지 않는 결정도 정상적인 종료입니다. P5·개발 단계 체크를 채울 필요가 없습니다.

Codex에 보낼 요청문

표시한 역할의 Codex 대화 입력칸

```
내 결정은 [기존 도구 사용 또는 보류]야. 이유와 실제 다음 행동·담당·재검토 날짜 또는 조건을 docs/research/04-decision.md
에 저장해줘. docs/STATE.md에도 개발하지 않고 마친 상태를 연결해. 지금은 개발 인계 문서나 코드를 만들지 마.
```

기준과 확인 범위

2026-11-02 현재 설치된 Herdr app 0.1.11, herdr-core 0.1.16, Codex CLI 0.166.0의 도움말·키 설정·앱 제공 문서와 공식 자료를 대조했습니다. A·B·C 배치는 설명용입니다. 실제 제품 구현·계정 로그인·재부팅 복구·운영 배포를 이 안내서 제작 과정에서 수행한 것은 아닙니다.

- Herdr app 공식 저장소 — space·tab·pane, 창 닫기와 터미널 종료의 차이
- tmux 공식 시작 안내 — 한 화면을 나누는 단위는 pane
- OpenAI Docs · CLI 명령 — resume·fork·new·rename·compact·status
- OpenAI Docs · 긴 작업의 대화 관리 — 같은 결과는 이어가기, 독립된 작업은 문서로 분리
- 설치본 근거: ~/.config/herdr-app/keys.json, AGENTS.md, mods/files/main.js, herdr-app commands, codex --help·resume --help·fork --help.

세션을 나누는 기준과 문서 이름은 이 안내의 권장 운영 방식이며 제품이 강제하는 기능이 아닙니다. 기존 프로젝트 지침·문서·기술을 우선합니다. 앱 설정·개인 설치 스킬은 변경하지 않았습니다.

체크 상태는 이 브라우저에만 저장합니다. 체크가 실제 검사나 배포를 실행하지 않습니다. · 인터넷 없이 이 HTML을 열어 읽을 수 있습니다.